PROJECT DEVELOPMENT PHASE

🧹 Code-Layout, Readability & Best Practices

📅 DATE 7/9/2026	👨‍🌾 NAME K.Hemanth Kumar	🌱 PROJECT OptiCrop – Smart Agricultural Production Optimization Engine	🏆 MAX MARKS 3 Marks

📦 Overview

This document describes how the OptiCrop codebase is organized and the coding conventions followed to keep it readable and maintainable.

✅ Practices Followed

ASPECT	PRACTICE FOLLOWED
File organization	Code is separated into data/, model/, and app/ folders so data generation, model training, and the web app each live in their own space.
Naming conventions	snake_case for variables and functions; descriptive constant names such as FEATURE_COLUMNS and CROP_INFO.
Comments & docstrings	Each script opens with a module-level docstring explaining its purpose and the pipeline steps inside it.
Modularity	Dataset generation (generate_dataset.py), model training (train_model.py), and the Flask app (app.py) are kept in separate files.
Consistency	PEP 8 style formatting is applied throughout: consistent indentation, spacing, and line length.
Reusability	Shared constants like FEATURE_COLUMNS and the CROP_INFO dictionary are defined once and reused across the app.
Error handling	Form input parsing in app.py is wrapped in a try/except block so invalid input doesn't crash the app.